

Литература

1. Страуструп Б. Язык программирования C++. – М.: Бином, 2011. – 1136 с.
2. Страуструп Б. Программирование. Принципы и практика с использованием C++. – М.: Вильямс, 2018. – 1328 с.
3. Подбельский В.В. Стандартный Си++ : учебное пособие. - М.: Финансы и статистика, 2014.- 687 с.
4. Джосаттис Н.М. Стандартная библиотека C++: справочное руководство, 2-е изд. – М.: ИД Вильямс, 2017. – 1136 с.
5. Уильямс Э. Параллельное программирование на C++ в действии. Практика разработки многопоточных программ.- М.: ДМК Пресс, 2016.-672 с.
6. Мэйерс С. Эффективное использование C++. 55 верных советов улучшить структуру и код ваших программ. – М.: ДМК Пресс, 2014. – 300 с.
7. Быков А.Ю. Решение задач на языках программирования Си и Си++: методические указания к выполнению лабораторных работ. – М.: МГТУ им. Н.Э. Баумана, 2017. - 244с. Режим доступа: <https://bmstu.press/catalog/item/4673>

Тема № 10. Основы объектно-ориентированного программирования (ООП) в Си++

Введение. Историческая справка

Язык Си++ появился в 80-х годах первоначально назывался «Си с классами», название Си++ появилось в 1983 г. (в Си существует операция ++ (инкремент – увеличение на 1). Один из создателей Бьёрн Страуструп.

Стандарты языка:

- ISO/IEC 14882:1998 “Standard for the C++ Programming Language”
- ISO/IEC 14882:2003 (исправления)
- C++ Technical Report 1 (TR1) 2005 г. ISO/IEC TR 19768, библиотека расширений C++ (проект документа)
- ISO/IEC 14882:2011 (C++11)
- C++ ISO/IEC JTC1 («International Standard ISO/IEC 14882:2014(E) Programming Language C++») (C++ 14)
- ISO/IEC 14882:2017 (C++ 2017)
- ISO/IEC 14882:2020 (C++ 2020)
- C++23 – ожидаемый стандарт языка программирования C++

Некоторые новшества в языке

1. При определении переменной не обязательно указывать тип, если он определяется по инициализатору, указывается тип auto.

auto a=10; auto x = 10.5;

2. C++11 вводит ключевое слово constexpr, которое позволяет пользователю

гарантировать, что или функция или конструктор объекта возвращает константу времени компиляции.

```
constexpr int GiveFive() { return 5; }  
int some_value[GiveFive() + 7];
```

3. C++11 позволяет вызывать одни конструкторы класса из других

4. Цикл for по набору значений (range base loop):

```
int m[] = { 1, 4, 10, 20, 8 };  
for (int k : m) // Для каждого k в массиве m  
cout << endl << k; // Печатается весь массив  
double X[] = { 1.3, 4.5, 5.6, 5.7 };  
for (double y : X) cout << endl << y;
```

5. Новый способ задания недействительного указателя (C++11):

```
int *p=nullptr;
```

6. Правосторонняя ссылка (ссылка на временный объект) T && (C++ 11).

7. Введено пространство имен - это группа имен, в которой имена не совпадают.

```
namespace name
```

```
{  
    // объявления и определения имен
```

```
int a;
```

```
}
```

За пределами пространства имен обращаемся к именам или *name::a=10;*

Пространства имен могут быть вложенными, в этом случае обращаемся к объекту (переменной, функции, классу и др.): *name1::name2::name3::a=10;*

Или используем директиву:

```
using namespace name;
```

Далее просто по имени *a=10;*

8. Заголовочные файлы без расширения h.

stdio.h – старый cstdio – новый

9. Целые константы в двоичной форме (C++ 14): префиксы 0b или 0B *int a = 0b11100011;*

10.1. Понятие об ООП

Объектно-ориентированное программирование

Суть – вводится новый тип – класс, включающий данные (поля класса) и методы (функции) для обработки этих данных. Переменные этого типа называются объектами.

Основные принципы ООП:

- Инкапсуляция (объединение в одном типе данных и методов)
- Наследование классов (объявление новых классов на базе существующих)
- Полиморфизм (переопределение методов в классах-наследниках)

10.2. Классы и объекты в Си++

Классы

```
<Ключевое слово> <Имя_класса>
```

```
{
```

```
    <список компонент>
```

```
};
```

Ключевые слова:

- struct
- class
- union

Класс это структура, в которую введены методы для обработки полей.

Объекты (переменные класса)

<Имя_класса> <Имя_объекта1>,...<Имя_объекта_N>;

Обращение к полям и методам класса внутри методов класса просто по имени, а за пределами класса через имя объекта и операцию «.» или через имя указателя на объект и операцию «->». Каждый объект класса имеет в оперативной памяти свои копии полей класса.

```
struct A { int i; void print() { printf("i=%d", i); } };
```

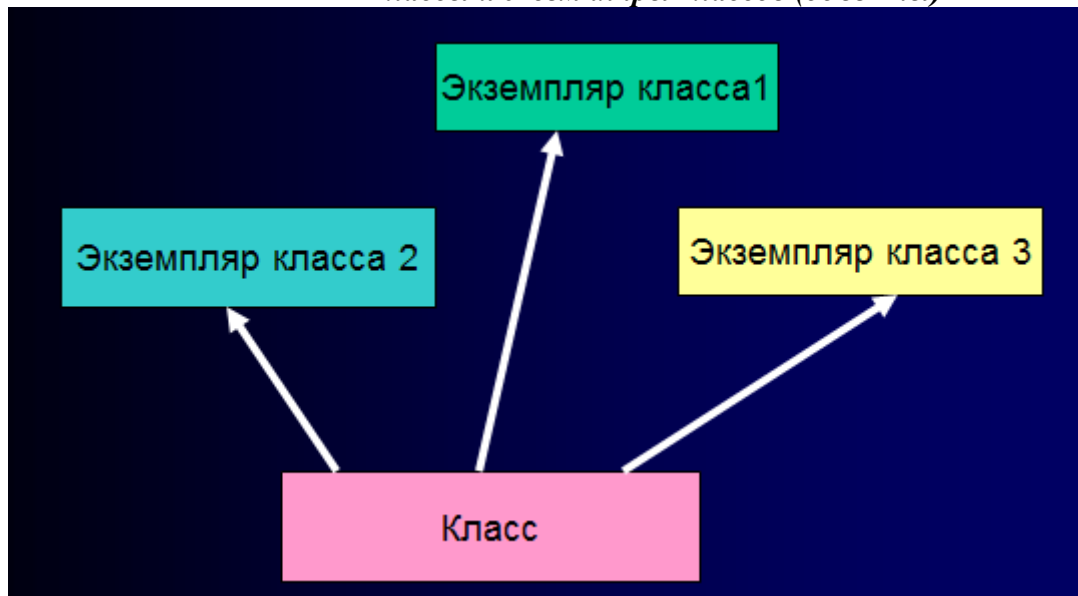
```
A a1; A *pA=&a1;
```

```
a1.i=10; a1.print(); pA->i=10; pA->print();
```

```
a1.A::i=10; a1.A::print();
```

```
pA->A::i=10; pA->A::print();
```

Классы и экземпляры классов (объекты)



10.3. Доступность компонент класса

Свойство доступности определяет возможность доступа к полям и методам за пределами класса (через имя объекта или через указатель на объект, или в производном классе).

Существуют три статуса доступа:

- public (полностью доступны за пределами класса);
- private (не доступны за пределами класса можно обращаться к компонентам только в методах своего класса);
- protected (доступны только в своем классе и в производных классах).

По умолчанию, если класс определен с ключевым словом struct, то все компоненты имеют статус доступа public. Если с ключевым словом union, тоже public, но все поля

каждого объекта располагаются в памяти, начиная с одного адреса. Если класс определен с ключевым словом `class`, то все поля и методы по умолчанию имеют статус доступа `private`.

Статус доступа можно изменить с помощью соответствующих модификаторов.

```
struct A
{
    ..... // Статус доступа public
private:
    ..... // Статус доступа private
protected:
    ..... // Статус доступа protected
};
class B
{
    ..... // Статус доступа private
public:
    ..... // Статус доступа public
protected:
    ..... // Статус доступа protected
};
```

10.4. Основные элементы класса

10.4.1. Компонентные данные и функции (методы) класса

```
struct Complex // Класс – комплексное число
{
    double real, image; // Поля класса
    void define(double re = 0.0, double im = 0.0) // Определение метода
        // внутри класса, метод будет подставляемым
    {
        real = re; image = im; // Обращение к полям внутри метода
    }
    void print(); // Описание метода
};
void Complex::print() // Определение метода за пределами класса
{
    cout<<"real="<<real<<" image="<< image<<endl;
}
```

Возможна перегрузка методов.

Чаще всего класс объявляется в 2-х файлах:

- заголовочный файл с расширением `.h`, содержит описание класса с заголовками методов;
- файл реализации с расширением `.cpp`, содержит определения методов за пределами класса.

При необходимости использовать класс в каком-либо файле проекта, к этому файлу требуется подключить заголовочный файл с описанием класса.

Допустимо объявление полей класса с инициализацией.

Инициализация выполняется до выполнения конструктора. Функция класса может иметь модификатор `const`, это означает, что она не может изменять поля класса, кроме полей с модификатором `mutable`

```
class point
{
```

```

    int x=-10;
    mutable int y=-10;

public:

void print() const;

.....

};

void point::print() const // Определение метода за пределами класса с модификатором const
{
    cout<<"\nx="<<x<<" y="<<y;
    y = 10; // Было бы запрещено, если бы y был без mutable
}

```

10.4.2. Конструктор класса

Конструктор класса специальный блок операторов (инструкций), вызываемый при создании объекта. Назначение: присвоение начальных значений полям, выделение памяти, открытие файлов, сетевых соединений и т.п. Имя конструктора совпадает с именем класса, конструктор не имеет возвращаемого значения. Возможна перегрузка конструкторов. Конструктор может определяться как внутри класса, так и за пределами.

Формат определения конструктора внутри класса:

```

Имя_класса(Список_формальных_параметров)
{ Операторы_тела конструктора }

```

По умолчанию класс всегда имеет конструктор копирования вида `A(const A& a) {...}` (`A` – имя класса), создающий копию объекта (происходит копирование полей), и, если нет ни одного явного конструктора, то по умолчанию создается конструктор без параметров. Эти конструкторы можно переопределять. Конструктор копирования по умолчанию можно удалить, для этого в классе заголовок конструктора объявляется с ключевым словом `delete` без тела, например, `A(const A& a)=delete;`

Если нет явного конструктора копирования, то конструктор копирования по умолчанию будет удален при наличии явного конструктора перемещения (см. ниже) или оператора копирования с перемещением.

Примеры вызовов конструкторов:

```

A a1; A* pA=new A; // Вызываются конструкторы без параметров
A a2(3, 4); A * pA2=new A(3, 4);
// Вызываются конструкторы с 2-мя параметрами

```

Для конструктора с одним параметром можно использовать форму:

```

A a1=5; A a2=a1; вместо A a1(5); A a2(a1);

```

Ключевое слово `explicit` у конструктора с одним параметром запрещает это преобразование, допустима явная форма: `A a1(5); A a2(a1);`

Пример:

```

class point
{
    int x, y;
public:

```

```

    point(int, int); // Конструктор с 2-мя параметрами
    explicit point(int); // Конструктор с 1-м параметром
    point() : x(0), y(0) // Можно инициализировать поля так
    { // Конструктор без параметров
        // x=0; y=0; // Или так
        cout << endl << "point()";
    }
    void print();
    ~point()
    {
        printf("\nDestructor");
    }
};

point::point(int a, int b) // Определение конструктора за пределами класса
    : x(a), y(b) // Можно так
{
    // x=a; y=b; // Или так
}

point::point(int a) // Определение конструктора за пределами класса
{
    x=a; y=0;
}

void point::print() // Определение метода за пределами класса
{
    cout<<"\nx="<<x<<" y="<<y;
}

int main(int argc, char* argv[])
{
    point *pP=new point[10]; // Массив указателей на объекты, создается 10 объектов,
вызываются конструкторы без параметров
    point p1(12, 13);
    p1.print();
    point p2; // Вызов конструктора без параметров
    point p3=100; // point p3(100); Вызов конструктора с 1-м параметром Ошибка, надо
        point p3(100); так как есть explicit у конструктора
    p2.print();
    p3.print();
    point p4=p1; // point p4(p1); Вызов конструктора копирования
    p4.print();
    delete [] pP;
// free(pP); Так не верно
    return 0;
}

```

10.4.3. Деструктор класса

Деструктор — специальный блок операторов (инструкций), служащий для деинициализации объекта (освобождение памяти, закрытие файлов и т.п.).

Вызывается автоматически при удалении объекта, например, оператором delete или при выходе из блока, в котором существует объект. Не имеет возвращаемого значения и параметров. Может определяться как внутри класса, так и за пределами.

```

~имя_класса()
{
    тело_деструктора
}

```

```
}
```

10.4.4. Конструктор перемещения (move constructor)

В отличие от конструктора копирования (этот конструктор «работает», как правило, быстрее), исходный объект уже не нужен или это временный объект, но его нужно привести в форму, подходящую для вызова деструктора, чтобы «не испортить» новый объект.

Пример:

```
Vec(Vec&& v) // Параметр - правосторонняя ссылка
// : p(v.p), len(v.len) Можно инициализировать так
{
    p = v.p;
    len = v.len;
    // Присвойте данным-членам исходного объекта значения по умолчанию.
    // Это не позволяет деструктору многократно освобождать память
    v.p = nullptr;
    v.len = 0;
}
```

Вызов конструктора: `Vec V2=std::move(V1);`

Пример:

```
#include <stdlib.h>
#include <iostream>

class Massiv
{
public:
    int *p=nullptr; // Изначально "пустой" объект
    int n=0;
    Massiv(int *pp, int nn);
    Massiv(const Massiv &M);
    // Конструктор перемещения
    Massiv(Massiv&& M); // Параметр - правосторонняя ссылка
    ~Massiv(); // Описание деструктора
    void print() const;
};

Massiv::~Massiv()
{
    if (p != nullptr) delete[]p;
    std::cout << "Destructor\n";
}

Massiv::Massiv(const Massiv & M)
{ // Новый конструктор копирования
    std::cout <<"Constr Copy\n";
    n = M.n;
    p = new int[n];
    for (int i = 0; i<n; i++)
        p[i] = M.p[i];
}
```

```

// Конструктор перемещения (вариант: делаем все явно)
/*Massiv::Massiv(Massiv&& M) // Параметр - правосторонняя ссылка
: p(M.p), n(M.n) // Можно инициализировать так
{
    std::cout << "Constr Move\n";
    //p = M.p; // Или инициализировать так
    //n = M.n;
    // Присвойте данным-членам исходного объекта значения по умолчанию. Это не
    позволяет деструктору многократно освобождать память
    M.p = nullptr;
    M.n = 0;
}*/

// Конструктор перемещения (более короткий вариант)
Massiv::Massiv(Massiv&& M) // Параметр - правосторонняя ссылка
{
    std::cout << "Constr Move\n";
    std::swap(p, M.p);
    std::swap(n, M.n);
}

Massiv::Massiv(int *pp, int nn)
{
    std::cout << "Constr\n";
    n = nn;
    p = new int[n];
    for (int i = 0; i < n; i++) p[i] = pp[i];
}

void Massiv::print() const
{
    if (p!=nullptr)
        for (int i = 0; i < n; i++)
            std::cout << p[i] << " ";
        else std::cout<<"Null Object";
    std::cout << "\n";
}

int main()
{
    int m[] = { 1, 2, 3, 4, 5, 6, 7 };
    Massiv M1(m, 7);
    M1.print();
    Massiv M2 = M1; // Massiv M2(M1); Вызывается конструктор копирования
    M2.print();
    M1.p[0] = 1000;
    M1.print();
    M2.print();
    {
        Massiv M3 = std::move(M1); // Вызов конструктора перемещения
        M3.print();
    }
}

```

```

    } // При выходе из блока возникла бы проблема потерянной ссылки (указателя)
или утечки памяти, если бы не было деструктора
    Massiv *pM = new Massiv(M1); // Конструктор копирования
    delete pM; // Автоматически вызывается деструктор
    return 0;
}

```

Если нет явного конструктора перемещения и копирования, то конструктор перемещения создается по умолчанию. Если есть явный конструктор копирования, то конструктор перемещения не создается, вместо него используется этот явный конструктор копирования. Конструктор перемещения также можно явно удалить, для этого в классе заголовок конструктора объявляется с ключевым словом **delete** без тела, например, `A( A&& a)=delete;`

10.5. Указатель *this*

Иногда при вызове нестатического метода требуется обращаться к адресу объекта, для которого вызывается метод. Но когда программист пишет код метода класса, объектов еще не существует. Для этих целей любой нестатический метод и конструкторы (деструктор) неявно (по умолчанию) получают указатель *this* – указатель на объект класса, для которого вызывается метод.

Первый случай использования this

```

class point {
    int x,y;
public:
    point(int x, int y)
    {
        this->x=x; this->y=y;
    }
    ....
};

```

Второй случай использования this

```

class List // Линейный односвязный список
{
    int x; // инф. поле
    List *pNext; // Указатель на след. элемент
public:
    List(int x)
    {
        this->x=x;
    }
    void add(List *&pF) // Добавить элемент в начало списка
    {
        pNext=pF;
        pF=this; // Текущий элемент будет первым
    }
}

```

```
.....  
};
```

Пример:

```
#include <iostream>  
using namespace std;  
class List // Линейный односвязный список  
{  
    int x; // инф. поле  
    List *pNext; // Указатель на след. элемент  
public:  
    List(int x)  
    {  
        this->x=x;  
    }  
    void add(List *&pF) // Добавить элемент в начало списка  
    {  
        pNext=pF;  
        pF=this; // Текущий элемент будет первым  
    }  
    static void print(List *pF)  
    {  
        List *pi=pF;  
        while(pi)  
        {  
            cout<<endl<<pi->x;  
            pi=pi->pNext;  
        }  
    }  
};  
  
int main(int argc, char* argv[])  
{  
    List *pF=0, *p;  
    for(int i=0; i<10; i++)  
    {  
        p=new List(i+1);  
        p->add(pF);  
    }  
  
    List::print(pF);  
    return 0;  
}
```

10.6. «Друзья» классов

Существует способ доступа к защищенным или собственным полям и методам класса за пределами класса. С помощью «друзей» класса (дружественных функций или дружественных классов). Они должны быть описаны внутри класса с модификатором friend.

10.6.1. Дружественные функции класса

Особенности дружественных функций (ДФ):

- ДФ не получает указатель this, т.к. она не принадлежит классу;
- дружественная функция получает доступ к объекту через формальный параметр, объект класса передается по указателю или по ссылке;
- место размещение прототипа ДФ внутри класса безразлично, на нее не

- распространяется действие модификаторов доступа;
- ДФ может быть компонентной функцией другого класса;
- ДФ может быть дружественной по отношению к нескольким классам.

10.6.2. Дружественные классы

```
class C1 { friend class C2; ... };
class C2 { .... }
```

Все функции класса C2 являются дружественными по отношению к классу C1.

Пример:

```
#include <iostream>
using namespace std;
class point
{
    int x, y;
public:
    point(int a, int b)
    {
        x=a; y=b;
    }
    friend void print(point &p); // Заголовок ДФ внутри класса
// friend void A::fun(point &p); // Если ДФ принадлежит классу А
};

void print(point &p) // Дружественная функция
{
    cout<<endl<<"x="<< p.x<<" y="<<p.y; // Имеет доступ к закрытым полям (private)
}
int main(int argc, char* argv[])
{
    point p1(3, 5);
    print(p1); // Вызов ДФ
    return 0;
}
```

10.7. Перегрузка стандартных операций

Перегрузка операций в языке Си++ это возможность распространения действия стандартных операций на операнды, для которых эти операции первоначально не предназначались. Это возможно, если хотя бы один из операндов является объектом класса, для этого создает специальная, так называемая, оператор- функция.

Формат оператор- функции

```
<тип_возвращаемого_значения> operator <знак_операции> (спецификация_параметров)
{
    операторы_тела_функции
}
```

Два способа перегрузки:

- оператор-функция определяется как функция не принадлежащая классу, при необходимости может быть дружественной;
- оператор-функция определяется как функция класса.

Особенности перегрузки операций:

- можно перегружать только стандартные операции, например, нельзя перегрузить **;
- не допускают перегрузки '.', '.*', '?:', '::', 'sizeof', '#', '##';
- сохраняется арность операций;
- бинарная операция перегружается либо как функция, не принадлежащая классу с двумя параметрами, один обязательно объект (ссылка на объект) класса, или как функция класса с одним параметром, первым операндом операции выступает объект класса, для которого вызывается функция;
- бинарные операции '=', '[]', '->' должны обязательно определяться как компонентные функции класса;
- унарная операция перегружается либо как функция, не принадлежащая классу с одним параметром - объектом (ссылкой на объект) класса, или как функция класса без параметров, операндом операции выступает объект класса, для которого вызывается функция.

Примечание 1. Исключение составляет перегрузка операций ++ и --, чтобы отличить постфиксную форму от префиксной, вводится формальный параметр типа int.

Пример объявления операторов-функций внутри класса:

```
Vector & operator++() // Префиксная форма операции ++  
  
Vector operator++(int) // Постфиксная форма операции ++ всегда есть формальный (фиктивный)  
                        // параметр типа int
```

Пример объявления операторов-функций за пределами класса:

```
Vector & operator++(Vector & v) // Префиксная форма операции ++  
  
Vector operator++(Vector & v, int) // Постфиксная форма операции ++ всегда есть формальный  
                                   // (фиктивный) параметр типа int
```

В постфиксной форме вначале с помощью конструктора копирования создаем новый объект, в котором сохраняется начальное (старое) состояние текущего объекта, затем текущий объект изменяется, и возвращается созданный объект.

Примечание 2. По аналогии с конструкторами копирования и перемещения по умолчанию существуют в классе операторы присваивания с копированием и присваивания с перемещением. Правила создания и использования аналогичные. Эти операторы можно переопределять или удалять, например,

```
A & operator=(const A & Ob) = delete;
```

```
A & operator=(A && Ob) = delete;
```

Пример:

```
#include <iostream>  
using namespace std;  
class Vector  
{  
    int* p = nullptr;  
    int n = 0;  
public:
```

```

Vector() // "Пустой" объект
{
    n = 0;
    p = nullptr;
}
~Vector() {
    if (p != nullptr) delete[] p;
    cout << "Destructor" << endl;
} // Деструктор
Vector(int* p, int n) // Создаем вектор на основе обычного массива
{
    this->n = n;
    this->p = new int[n];
    for (int i = 0; i < n; i++) this->p[i] = p[i];
}
Vector(int n) : n(n) // Выделяем память без инициализации
{
    p = new int[n];
}
// Перепускаем операции обращения по индексу
int& operator[](int index) // ссылка позволяет изменять значение элемента
{
    return p[index]; // Возвращаем элемент по индексу
}
int operator[](int index) const // Изменять значение нельзя
{
    return p[index]; // Возвращаем элемент по индексу
}
Vector& operator++() // Префиксная форма операции ++
{
    for (int i = 0; i < n; i++) ++p[i];
    return *this;
}
Vector operator++(int) // Постфиксная форма операции ++ всегда есть формальный
(фиктивный) параметр типа int
{
    Vector temp(*this); // Создаем новый объект (старое состояние текущего объекта)
    for (int i = 0; i < n; i++) ++p[i]; // Инкремент текущего объекта
    return temp; // Возвращаем старое состояние объекта
}
Vector(const Vector& V) // Конструктор копирования
{
    // Задаем новый объект
    n = V.n;
    p = new int[n];
    for (int i = 0; i < n; i++) p[i] = V.p[i];
    cout << "Constructor Copy" << endl;
}
Vector(Vector&& V) // Конструктор перемещения
{
    // Короткая форма
    swap(p, V.p);
    swap(n, V.n);
    cout << "Constructor Move" << endl;
}
Vector& operator=(const Vector& V2) // Оператор присваивания с копированием, пара-
метр второй операнд, первый операнд – объект текущего класса
{
    if (this != &V2) // Запрет копирования объекта самого в себя
    {

```

```

        if (p != nullptr) delete[] p; // Очищает (удаляет) старый объект,если он
есть
        // Задаем новый объект
        n = V2.n;
        p = new int[n];
        for (int i = 0; i < n; i++) p[i] = V2.p[i];
    }
    cout << "operator= copy" << endl;
    return *this; // Возвращаем текущий объект
}
Vector& operator=(Vector&& V2) // Оператор присваивания с перемещением, Параметр
второй операнд, первый операнд – объект текущего класса
{
    if (this != &V2) // Запрет перемещения объекта самого в себя
    {
        /* // Все делаем явно
есть
        if (p != nullptr) delete[] p; // Очищает (удаляет) старый объект,если он

        // Задаем новый объект
        n = V2.n;
        p = V2.p;
        V2.n = 0; V2.p = nullptr;*/
        // Короткая форма
        swap(p, V2.p);
        swap(n, V2.n);
    }
    cout << "operator= move" << endl;
    return *this; // Возвращаем текущий объект
}
// Умножение вектора на число
// на выходе новый вектор
Vector operator*(int x) const
{
    Vector V(n);
    for (int i = 0; i < n; i++)
        V[i] = p[i] * x;
    return V;
}
friend ostream& operator<<(ostream& out, const Vector& v); // Перегрузка операции
вставки в поток
friend Vector operator*(int x, const Vector& v2);
friend double operator*(const double* p1, const Vector& Ob2); // Чтобы получить до-
ступ к закрытым полям класса
// friend Vector & operator++(Vector & V); // Префиксная форма операции ++
// friend Vector operator++(Vector & V, int); // Постфиксная форма операции
++ всегда есть формальный (фиктивный) параметр типа int
};

ostream& operator<<(ostream& out, const Vector& v) // Перегрузка операции вставки в
поток
{
    if (v.p != nullptr) for (int i = 0; i < v.n; i++)
        out << v.p[i] << ' ';
    else out << "Empty";
    out << endl;
    return out;
}
// Умножение числа на вектор, так как 1 операнд не объект класса, то оператор- функция
не принадлежит классу
Vector operator*(int x, const Vector& v2)

```

```

{
    Vector V(v2.n);
    for (int i = 0; i < v2.n; i++)
        V[i] = x * v2[i];
    return V;
}
// Скалярное произведение векторов
// Оператор-функция обязательно не принадлежит классу, так как первый операнд не объект класса
double operator*(const double* p1, const Vector& Ob2)
{
    double sum = 0;
    for (int i = 0; i < Ob2.n; i++)
        sum += p1[i] * Ob2.p[i];
    return sum;
}

// Пример перегрузки операторов ++ вне класса
/*Vector & operator++(Vector & V) // Префиксная форма операции ++
{
    for (int i = 0; i < V.n; i++) ++V.p[i];
    return V;
}
Vector operator++(Vector & V, int) // Постфиксная форма операции ++ всегда есть формальный (фиктивный) параметр типа int
{
    Vector temp(V); // Создаем новый объект (старое состояние текущего объекта)
    for (int i = 0; i < V.n; i++) ++V.p[i]; // Инкремент текущего объекта
    return temp; // Возвращаем старое состояние объекта
}*/
int main()
{
    int m1[] = { 1, 2, 3, 4, 5 };
    Vector V1(m1, 5); // Создаем объект на основе обычного массива
    cout << V1;
    V1[0] = 100;
    cout << V1;
    Vector V2;
    V2 = 2 * V1; // V2=operator*(2, V1);
    cout << V2;
    Vector V3;
    //V3 = move(V1++); // Чтобы не создавать лишние объекты используем оператор перемещения
    V3 = V1++; // Здесь тоже вызывается оператор = с перемещением Чтобы не создавать лишние объекты
    cout << V1 << V3;
    return 0;
}

```

Правило трёх (также известное как «Закон Большой Тройки» или «Большая Тройка») - правило в C++, гласящее, что если класс или структура определяет один из следующих методов, то они должны явным образом определить все три метода:

- Деструктор
- Конструктор копирования
- Оператор присваивания копированием

С выходом одиннадцатого стандарта правило расширилось и теперь называется правило пяти. Теперь при реализации конструктора необходимо реализовать:

- Деструктор

- Конструктор копирования
- Оператор присваивания копированием
- Конструктор перемещения
- Оператор присваивания перемещением

10.8. Статические компоненты класса

Статические поля

Память под обычные поля (нестатические) выделяется при создании объекта, каждый объект класса имеет свои копии обычных полей.

Статические поля объявляются в классе с модификатором `static`, память под статические поля выделяется при определении класса, и не выделяется при создании объектов. Статические поля существуют в единственном экземпляре независимо от того, сколько объектов создано, т.е. это поля класса, они общие для всех объектов. К статическим полям за пределами класса можно обращаться через имя объекта, как к обычным полям, но чаще к ним обращаются через имя класса (это можно делать даже когда объекты класса не созданы). Статические поля обязательно требуют инициализации (за пределами класса).

Статические методы

На статические поля распространяются модификаторы статуса доступа. Для доступа к собственным (`private`) или защищенным (`protected`) статическим полям за пределами класса служат открытые статические методы, которые определяются с модификатором `static`.

К статическим методам за пределами класса обращаются как к статическим полям через имя объекта или чаще через имя класса. Внутри статических методов можно обращаться только к статическим полям или вызывать другие статические методы. К обычным полям и методам обращаться нельзя, так как при вызове статического метода объекты могут быть еще не созданы (статический метод относится ко всему классу).

Пример:

```
#include <iostream>
using namespace std;

class A
{
    static int N; // Счетчик созданных объектов
public:
    A()
    {
        N++;
    }
    ~A() { N--; }
    static int getN();
};

int A::getN()
{
    return N;
}

int A::N=0; // Обязательная инициализация статич поля
```

```

int main(int argc, char* argv[])
{
    cout << "N=" << A::getN();
    A a1, a2, a3, a4;
    cout<<"\nN="<< a1.getN();
    {
        A a5;
        cout << "\nN="<< a5.getN();
    }
    cout << "\nN="<< A::getN();
    return 0;
}

```

10.9. Указатели на компоненты класса

Операции ‘.’*’ и ‘->*’ предназначены для работы с указателями на компоненты класса. До использования указателя его необходимо соответствующим образом настроить.

10.9.1. Указатели на компонентные функции (методы)

Формат определения указателя на функцию:

`<тип_возвр_значения> (<имя_класса>::*<имя_указателя>)(<спецификация_форм_параметров>);`

Настройка указателя:

`<имя_указателя>= &<имя_класса>::<имя_функции>;`

Вызов функции по указателю:

`(<имя_объекта>.*<имя_указателя>)(<параметры>);`

`(<имя_ук_на_об>->*<имя_указателя>)(<параметры>);`

10.9.2. Указатели на поля

Формат определения указателя на поле класса:

`<тип_поля> (<имя_класса>::*<имя_указателя>);`

Настройка указателя:

`<имя_указателя>= &<имя_класса>::<имя_поля>;`

Обращение к полю по указателю:

`<имя_объекта>.*<имя_указателя>=10;`

`<имя_ук_на_об>->*<имя_указателя>=10;`

Пример:

```

#include <iostream>
using namespace std;
struct point
{
    int x, y;
    void printX()
    {
        cout<<endl<<"x="<<x;
    }
    void printY()
    {
        cout<<endl<<"y="<<y;
    }
};

```

```

int main(int argc, char* argv[])
{
    void (point::*pF)()=&point::printX; // Указатель на функцию класса
    int point::*p=&point::x; // Указатель на поле класса
    point p1;
    // p1.x=5;
    // p1.y=10;
    p1.*p=5; // Обращение к полю x через указатель
    p=&point::y; // Указатель настраиваем на поле y
    p1.*p=10; // Обращение к полю y через указатель
    (p1.*pF)(); // Вызываем функцию printX через указатель
    pF=&point::printY; // Указатель на функцию класса настраиваем на функцию printY
    (p1.*pF)(); // Вызываем функцию printY через указатель

    return 0;
}

```

10.10 Локальные классы

Локальный класс – класс, определенный внутри блока.

- локальный класс не может иметь статических данных;
- его компонентные функции могут быть только встроенными;
- внутри класса можно использовать любые внешние переменные, функции, которые можно использовать внутри блока, за исключением переменных класса памяти auto.

10.11. Шаблоны классов

Шаблон класса задает семейство классов, в которых могут быть использованы разные типы.

```

template <список_параметров_шаблона>
<определение_шаблона_класса>

```

Имя параметра шаблона – это имя неизвестного заранее типа. При создании объекта указывается имя конкретного типа и по шаблону генерируется класс.

Существует библиотека стандартных шаблонов (STL- Standard Template Library), вошла в стандарт C++ 11

В ней шаблонами задаются контейнеры – предназначенные для хранения набора объектов в памяти (объекты могут быть любых типов). Контейнеры: vector, list, set, stack, queue и другие.

Пример:

```

#include <iostream>
using namespace std;
template <class T> // Шаблон класса вектор (массив)
// T - тип элементов вектора
class вектор
{
    T* data; // Указатель на массив
    int size; // Размерность
public:
    вектор(int n=10);
    ~вектор() { delete [] data; }
    T& operator[] (int i) // Оператор функция возвращает элемент по индексу
    {

```

```

        return data[i];
    }
    T getSum(); // Функция считает сумму элементов
};
// Внешнее определение конструктора
template <class T> // Перед любым определением метода за пределами класса
vektor<T>::vektor(int n)
{
    data=new T[n]; size=n;
}

// Внешнее определение функции
template <class T> // Перед любым определением метода за пределами класса
T vektor<T>::getSum()
{
    T sum=0;
    for(int i=0; i<size; i++)
        sum+=data[i];
    return sum;
}
int main()
{
    vektor<int> X(5); // Вектор элементов типа int
    X[1]=101; // Вызов оператор функции
    cout<<"x[1]="<< X[1];
    vektor<double> Y(10); // Создается класс вместо T подставляется double
    Y[0]=10.51; // Вызов оператор функции
    cout << "\nY[0]=" << Y[0];
    for(int i=0; i<10; i++) Y[i]=i;
    cout << "\nSum="<< Y.getSum();
    return 0;
}

```

Примечание. Вместо ключевого слова `class` можно перед параметром шаблона использовать ключевое слово `typename`

```

template <typename T> // Шаблон класса вектор (массив)
// T - тип элементов вектора
class vektor
{
    ...
};

```